Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4
По дисциплине: “СПП”

Выполнил:
Студент 3 курса
Группы ПО-12
Мартынюк В. В.
Проверила:
Кулик А. Д.

Брест 2026

Цель работы: научиться работать с Github API, приобрести практические навыки написания программ для работы с REST API или GraphQL API

10. Автоматический анализ самых активных контрибьюторов в репозитории

Условие:

Напишите Python-скрипт, который анализирует самых активных контрибьюторов в указанном GitHub-репозитории за определённый период.

Что нужно сделать:

1. Запросить у пользователя:

 Репозиторий (owner/repo, например, tensorflow/tensorflow).

 Период анализа (за неделю, за месяц, за год).

 (Опционально) Минимальное количество коммитов для попадания в рейтинг.

2. Использовать GitHub API для получения списка контрибьюторов и их активности:

 Количество коммитов.

 Количество добавленных и удалённых строк кода.

 Количество открытых и закрытых pull requests.

 Количество открытых и закрытых issues.

 Количество комментариев к PR и issues.

3. Определить ТОП-5 самых активных контрибьюторов.

4. Построить график активности (matplotlib, seaborn).

Решение:

```
import requests
import matplotlib.pyplot as plt
import numpy as np
from datetime import datetime, timedelta
import time

HEADERS = {}

def get_user_input():
    repo = input("Введите репозиторий (owner/repo): ").strip()
    period = int(input("Выберите период (7 / 30 / 365 дней): ").strip())
    min_commits_input = input("Минимальное количество коммитов для рейтинга (Enter,
чтобы пропустить): ").strip()
    min_commits = int(min_commits_input) if min_commits_input.isdigit() else 0
    return repo, period, min_commits

def fetch_contributors(repo):
    url = f"https://api.github.com/repos/{repo}/contributors"
    params = {"anon": "false", "per_page": 100}
    contributors = []
    page = 1
```

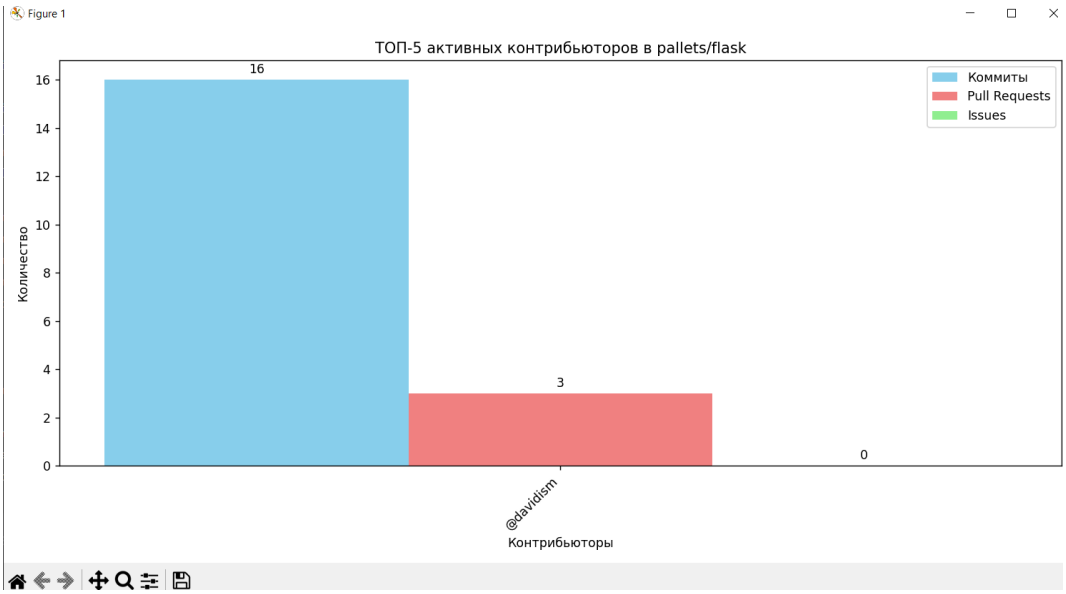
```
    while True:
        try:
            params['page'] = page
            response = requests.get(url, headers=HEADERS, params=params)
            response.raise_for_status()
            data = response.json()
            if not data:
                break
            contributors.extend([c for c in data if c.get('login')])
            page += 1
            time.sleep(0.1)
        except requests.exceptions.RequestException as e:
            print(f"Ошибка при получении списка контрибьюторов: {e}")
            break
    return {c['login']: c['contributions'] for c in contributors}

def fetch_commits_since(repo, since_date):
    url = f"https://api.github.com/repos/{repo}/commits"
    params = {"since": since_date, "per_page": 100}
    authors_commits = {}
    page = 1

    while True:
        try:
            params['page'] = page
            response = requests.get(url, headers=HEADERS, params=params)
            response.raise_for_status()
            data = response.json()
            if not data:
                break
            for commit in data:
                author_login = commit.get('author')
                if author_login and author_login.get('login'):
                    login = author_login['login']
                    authors_commits[login] = authors_commits.get(login, 0) + 1
            page += 1
            if page % 3 == 0:
                time.sleep(0.5)
        except requests.exceptions.RequestException as e:
            print(f"Ошибка при получении коммитов: {e}")
            break
    return authors_commits

def fetch_user_prs(repo, author, since_date):
    url = f"https://api.github.com/search/issues"
    query = f"repo:{repo} type:pr author:{author} created:>={since_date}"
    params = {"q": query}
    try:
        response = requests.get(url, headers=HEADERS, params=params)
        response.raise_for_status()
        data = response.json()
        return data['total_count']
    except:
        return 0
```

```
def fetch_user_issues(repo, author, since_date):
    url = f"https://api.github.com/search/issues"
    query = f"repo:{repo} type:issue author:{author} created:>={since_date}"
    params = {"q": query}
    try:
        response = requests.get(url, headers=HEADERS, params=params)
        response.raise_for_status()
        data = response.json()
        return data['total_count']
    except:
        return 0

def fetch_user_comments(repo, author, since_date):
    url = f"https://api.github.com/search/issues"
    query = f"repo:{repo} commenter:{author} updated:>={since_date}"
    params = {"q": query}
    try:
        response = requests.get(url, headers=HEADERS, params=params)
        response.raise_for_status()
        data = response.json()
        return data['total_count']
    except:
        return 0

def analyze_contributors(repo, period_days, min_commits):
    since = (datetime.now() - timedelta(days=period_days)).isoformat() + "Z"
    print(f"Анализ активности с {since}...")

    all_contributors = fetch_contributors(repo)
    active_committers = fetch_commits_since(repo, since)

    results = []
    for login, commits in active_committers.items():
        if commits < min_commits:
            continue

        print(f"Обработка {login}...")
        prs = fetch_user_prs(repo, login, since)
        issues = fetch_user_issues(repo, login, since)
        comments = fetch_user_comments(repo, login, since)

        total_activity_score = commits + prs + issues + comments

        results.append({
            'login': login,
            'commits': commits,
            'prs': prs,
            'issues': issues,
            'comments': comments,
            'score': total_activity_score
        })
        time.sleep(0.2)

    top_5 = sorted(results, key=lambda x: x['score'], reverse=True)[:5]
```

```
    return top_5

def plot_top_contributors(top_5, repo_name):
    if not top_5:
        print("Нет данных для построения графика.")
        return

    logins = [f"@{c['login']}" for c in top_5]
    commits = [c['commits'] for c in top_5]
    prs = [c['prs'] for c in top_5]
    issues = [c['issues'] for c in top_5]

    x = np.arange(len(logins))
    width = 0.25

    fig, ax = plt.subplots(figsize=(12, 6))
    rects1 = ax.bar(x - width, commits, width, label='Коммиты', color='skyblue')
    rects2 = ax.bar(x, prs, width, label='Pull Requests', color='lightcoral')
    rects3 = ax.bar(x + width, issues, width, label='Issues', color='lightgreen')

    ax.set_xlabel('Контрибьюторы')
    ax.set_ylabel('Количество')
    ax.set_title(f'ТОП-5 активных контрибьюторов в {repo_name}')
    ax.set_xticks(x)
    ax.set_xticklabels(logins, rotation=45, ha='right')
    ax.legend()

    ax.bar_label(rects1, padding=3)
    ax.bar_label(rects2, padding=3)
    ax.bar_label(rects3, padding=3)

    fig.tight_layout()

    filename = repo_name.replace('/', '_') + "_contributors.png"
    plt.savefig(filename, dpi=150)
    print(f"График активности сохранен в '{filename}'")
    plt.show()

def print_top_table(top_5):
    print("\n" + "="*80)
    print("ТОП-5 активных контрибьюторов:")
    print("="*80)
    for i, c in enumerate(top_5, 1):
        print(f"{i}. @{c['login']} - {c['commits']} коммитов, "
              f"{c['prs']} PR, {c['issues']} issues, {c['comments']} комментариев")
    print("="*80)

if __name__ == "__main__":
    repo, period, min_commits = get_user_input()
    top_contributors = analyze_contributors(repo, period, min_commits)

    if top_contributors:
        print_top_table(top_contributors)
        plot_top_contributors(top_contributors, repo)
    else:
```

```
print("Не найдено контрибьюторов, удовлетворяющих условиям.")
```

```
D:\LABS\CH\LB3\3\.venv\Scripts\python.exe D:\LABS\CHH\3\main.py
Введите репозиторий (owner/repo): pallets/flask
Выберите период (7 / 30 / 365 дней): 30
Минимальное количество коммитов для рейтинга (Enter, чтобы пропустить): 5
Анализ активности с 2026-02-17T19:56:28.331571Z...
Обработка davidism...

================================================================================
ТОП-5 активных контрибьюторов:
================================================================================
1. @davidism - 16 коммитов, 3 PR, 0 issues, 9 комментариев
================================================================================
График активности сохранен в 'pallets_flask_contributors.png'
```



Вывод: В результате выполнения лабораторной работы были закреплены знания объектно-ориентированного программирования Python при решении задач.